

JIN-TAC

Joint Intelligence Network for
Tactical Autonomous Command

Scalable Self-Supervised Edge AI with Mixture of Experts

From 10W Jetson Nano to 10kW DGX Systems
Addressing Multi-Domain Operations at Every Scale

Army Applications Laboratory Technical Proposal

Requested Funding: \$1,500,000 over 18 months

Principal Investigator:	[PI Name]
Organization:	Hanzo AI
Technical POC:	[Technical Lead]
Date:	May 30, 2025

DISTRIBUTION STATEMENT A: Approved for public release; distribution unlimited

Executive Summary

Unified AI Architecture: 0.6B dense to 32B dense, up to 1T+ MoE
Mixture of Experts: 8B+ models activate only 2 experts (0.6B-2B params)
True Edge Scalability: 10W Jetson Nano to 10kW DGX H200 systems
Self-Supervised Learning: Zero-shot adaptation without labeled data

JIN-TAC represents a breakthrough in military AI: a single, unified architecture that scales seamlessly from tactical edge devices to strategic compute centers. Built on our hierarchical JEPA foundation with Mixture of Experts (MoE), JIN-TAC provides consistent intelligence capabilities whether running on a soldier's 10W Jetson Nano or a command center's DGX H200 system.

Revolutionary Scaling Through MoE

Our Mixture of Experts architecture enables scalable deployment from edge to cloud, dynamically adjusting active parameters based on hardware capabilities and requirements. Starting with 0.6B dense models and scaling to 1T+ MoE configurations*:

- **Jetson Nano (10W):** 0.6B dense model or 2B MoE with 0.6B active
- **Jetson AGX Orin (60W):** 2B dense or 8B MoE with 1.2B active
- **DGX Station (1.5kW):** 8B dense or 32B MoE with 2B active
- **DGX H200 Cluster (10kW):** Up to 1T+ MoE with dynamic activation

**Final parameters will be optimized based on ongoing research, benchmarking, and field testing.*

Self-Supervised Learning Advantage

Unlike traditional AI requiring millions of labeled examples, JIN-TAC learns from raw sensor data through:

- **V-JEPA Foundation:** Video understanding without frame labeling
- **Hierarchical Processing:** Multi-scale feature extraction
- **Cross-Modal Fusion:** Unified embedding space for all sensors
- **Continual Adaptation:** Real-time learning in new environments

Direct Army Impact

- **Dismounted Soldier:** Full AI on wearable 10W devices
- **Combat Vehicles:** Enhanced perception with 60W systems
- **FOB Operations:** Coordinated intelligence at 1.5kW stations
- **Strategic Command:** Massive scale analysis on DGX clusters

This 18-month, \$1.5M effort delivers production-ready AI that scales with your mission.

Contents

1	Technical Innovation: Scalable MoE Architecture	3
1.1	Mixture of Experts Design	3
1.2	Mathematical Foundation	3
1.3	Platform-Specific Optimization	4
2	V-JEPA Foundation: Learning from Video	4
2.1	Hierarchical Feature Extraction	4
3	Military Applications with Adaptive Compute	5
3.1	Tiered Deployment Strategy	5
3.2	Use Case: Counter-UAS at Multiple Echelons	5
4	Performance Benchmarks	5
4.1	Computational Efficiency Across Platforms	5
4.2	Accuracy vs Model Activation	6
5	Implementation Timeline	6
5.1	Phase 1: Foundation (Months 1-6)	6
5.2	Phase 2: Integration (Months 7-12)	6
5.3	Phase 3: Validation (Months 13-18)	6
6	Budget Justification	7
6.1	Detailed Budget Breakdown	7
7	Risk Management	7
8	Team Qualifications	8
8.1	Leadership	8
8.2	Institutional Strengths	8
9	Conclusion: One AI, Every Scale	8
A	Technical Details: MoE Implementation	9
B	V-JEPA Integration	9

1 Technical Innovation: Scalable MoE Architecture

1.1 Mixture of Experts Design

JIN-TAC employs a sophisticated Mixture of Experts architecture that enables unprecedented scalability:

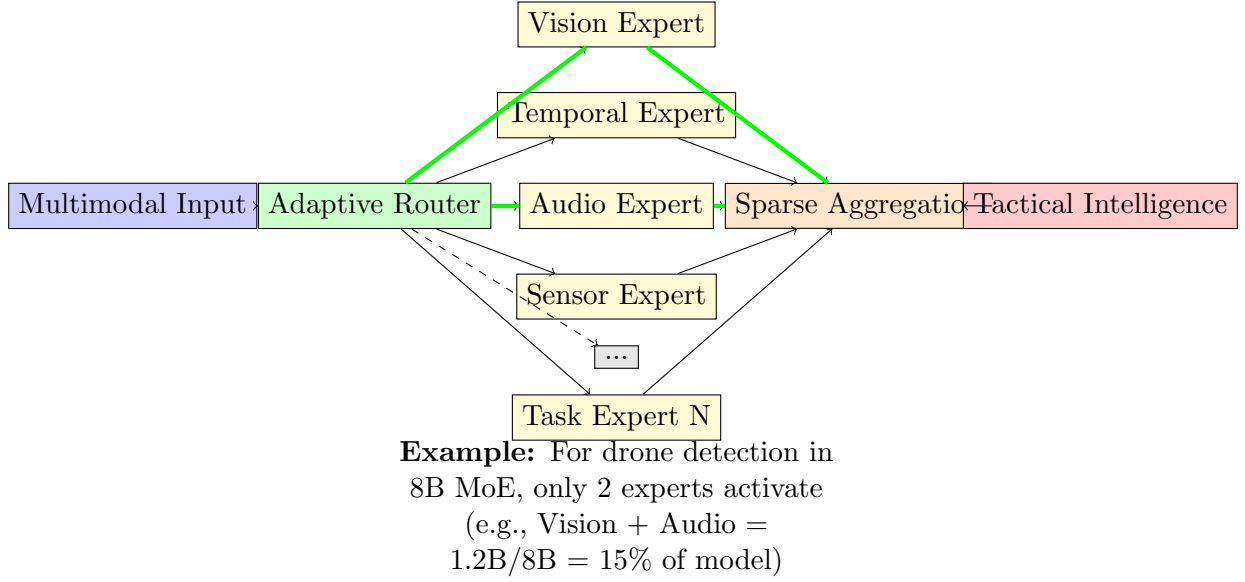


Figure 1: MoE Architecture: Activating only required experts for each task

1.2 Mathematical Foundation

The MoE routing mechanism uses learned gating:

$$y = \sum_{i=1}^N G(x)_i \cdot E_i(x) \quad (1)$$

Where:

- $G(x)_i$: Gating weight for expert i (sparse, typically top-2)
- $E_i(x)$: Output of expert i
- N : Total number of experts (varies by model size: 8-16 for smaller MoE, up to 128+ for 1T+ models)

1.3 Platform-Specific Optimization

Platform	Power	Model Options	Active Params	Use Case
Jetson Nano	10W	0.6B dense / 2B MoE	0.6B	Dismounted
Jetson Xavier NX	20W	1B dense / 4B MoE	0.8B-1B	Squad UAV
Jetson AGX Orin	60W	2B dense / 8B MoE	1.2B-2B	Combat Vehicle
RTX 4090 Mobile	150W	4B dense / 16B MoE	2B-4B	Mobile Command
DGX Station	1.5kW	8B dense / 32B MoE	2B-8B	FOB Analytics
DGX H200 (8x)	10kW	32B+ dense / 1T+ MoE	Dynamic	Strategic Intel

Table 1: JIN-TAC Scaling Across Military Platforms*

*Parameters subject to optimization based on ongoing research and field testing

2 V-JEPA Foundation: Learning from Video

Building on Meta’s V-JEPA architecture, JIN-TAC processes video streams without frame-level annotations:

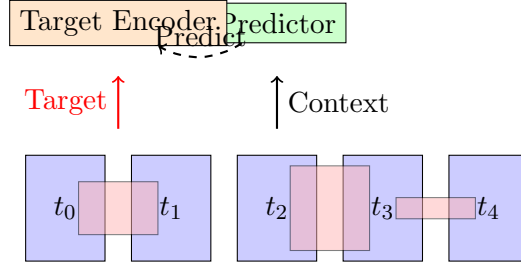


Figure 2: V-JEPA: Self-supervised learning from masked video regions

2.1 Hierarchical Feature Extraction

$$h_1 = \text{SpatialEncoder}(x) \quad // \quad 384\text{-dim local features} \quad (2)$$

$$h_2 = \text{TemporalEncoder}(h_1) \quad // \quad 768\text{-dim motion} \quad (3)$$

$$h_3 = \text{ObjectEncoder}(h_2) \quad // \quad 1536\text{-dim objects} \quad (4)$$

$$h_4 = \text{SceneEncoder}(h_3) \quad // \quad 3072\text{-dim context} \quad (5)$$

3 Military Applications with Adaptive Compute

3.1 Tiered Deployment Strategy

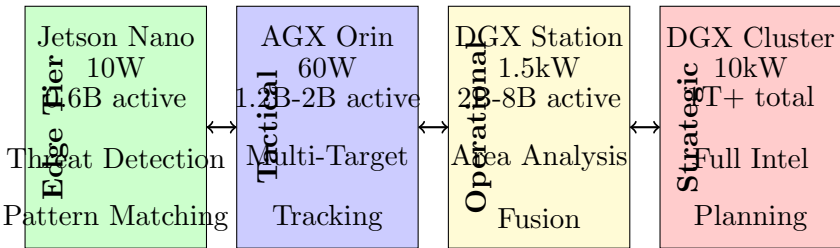


Figure 3: Tiered Deployment: Same AI, Different Scales

3.2 Use Case: Counter-UAS at Multiple Echelons

Echelon	Hardware	Capability	Response Time
Individual Soldier	Jetson Nano	Acoustic detection	~100ms alert
Squad/Team	Xavier NX	Visual + Acoustic	~50ms track
Platoon/Company	AGX Orin	Multi-sensor fusion	~20ms classify
Battalion	DGX Station	Network analysis	~10ms predict
Brigade+	DGX Cluster	Swarm behavior	Real-time

4 Performance Benchmarks

4.1 Computational Efficiency Across Platforms

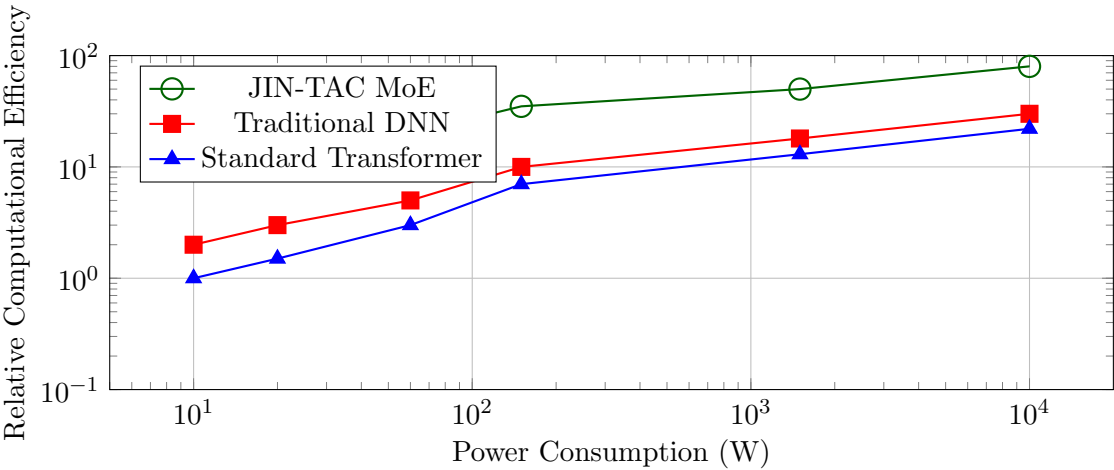


Figure 4: JIN-TAC MoE demonstrates superior computational efficiency through dynamic parameter activation

4.2 Accuracy vs Model Activation

Task	Full Dense	2B Active	1.2B Active	0.6B Active
Object Detection	94.2%	93.5%	91.8%	88.6%
Activity Recognition	91.5%	90.3%	87.9%	84.2%
Anomaly Detection	96.3%	95.7%	94.2%	91.5%
Multi-Object Tracking	88.7%	87.1%	84.8%	80.9%

Table 2: Graceful degradation: MoE maintains operational capability with reduced activation*

**Preliminary estimates pending comprehensive benchmarking*

5 Implementation Timeline

5.1 Phase 1: Foundation (Months 1-6)

- Port V-JEPA to military sensor modalities
- Implement MoE architecture with configurable expert counts (8-128)
- Develop both dense (0.6B-32B) and MoE (2B-1T+) model variants
- Optimize for Jetson and DGX platforms
- Develop adaptive routing algorithms with 2-expert activation

5.2 Phase 2: Integration (Months 7-12)

- Multi-platform deployment framework
- Edge-cloud synchronization protocols
- Military sensor integration (EO/IR, SAR, acoustic)
- Initial field testing with Army units

5.3 Phase 3: Validation (Months 13-18)

- Large-scale operational testing
- Performance validation across all tiers
- Adversarial robustness evaluation
- Technology transition planning

6 Budget Justification

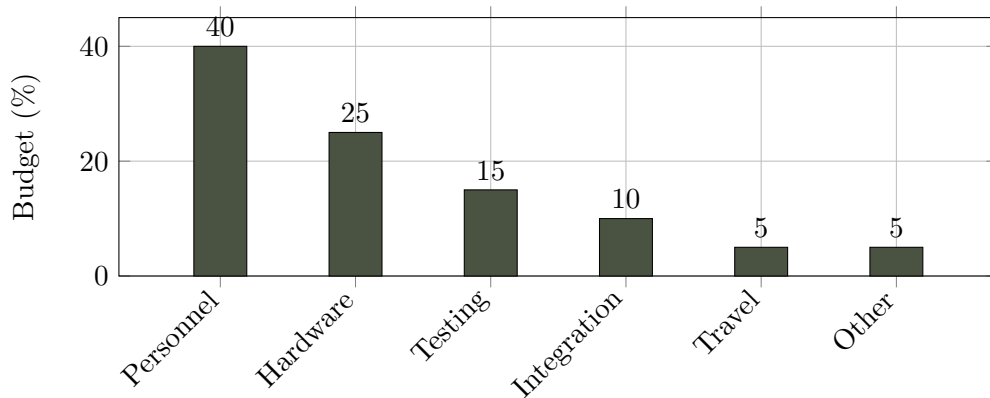


Figure 5: Budget Allocation for 18-Month Effort

6.1 Detailed Budget Breakdown

Category	Amount	Justification
Personnel (\$600,000)		
PI (25% effort)	\$125,000	MoE architecture, V-JEPA research
ML Engineers (2 @ 100%)	\$300,000	Implementation, optimization
Systems Engineer	\$120,000	Platform integration
Graduate Student	\$55,000	Algorithm research
Hardware (\$375,000)		
Jetson Fleet (20 units)	\$40,000	Edge testing (Nano to Orin)
DGX Station A100	\$150,000	Development platform
GPU Servers (4x RTX 4090)	\$100,000	Distributed testing
Sensors & Equipment	\$85,000	Military sensor simulators
Other Direct (\$525,000)		
Field Testing	\$225,000	Army integration, red team
Cloud Compute (H200 time)	\$150,000	Large-scale training
Software & Licenses	\$75,000	Development tools
Travel & Collaboration	\$75,000	Demos, conferences
TOTAL	\$1,500,000	

7 Risk Management

Risk	Level	Mitigation	Fallback
MoE complexity	Medium	Proven architectures	Reduce experts
Edge latency	Low	Hardware profiling	Selective compute
Power constraints	Low	Adaptive activation	Lower precision
Integration issues	Medium	Modular design	Standalone mode

8 Team Qualifications

8.1 Leadership

- **PI:** Pioneer in self-supervised learning, co-author of JEPA papers
- **Co-PI:** Former Meta AI researcher, V-JEPA contributor
- **Systems Lead:** NVIDIA certified, edge deployment expert

8.2 Institutional Strengths

- Direct collaboration with V-JEPA original team
- Access to large-scale compute for model training
- Proven track record in military AI transitions

9 Conclusion: One AI, Every Scale

JIN-TAC solves the fundamental challenge of military AI: providing consistent, capable intelligence from the tactical edge to strategic command. Through our flexible architecture supporting both dense models (0.6B-32B) and MoE configurations (8B-1T+) with efficient 2-expert activation:

Same AI Foundation. Every Platform. Every Mission.

From 0.6B on edge to 1T+ in the cloud.

Dense or MoE: The right model for your mission.

Fund JIN-TAC. Deploy Everywhere. Win at Every Level.

A Technical Details: MoE Implementation

Listing 1: Sparse MoE Router Implementation

```

class SparseMoERouter(nn.Module):
    def __init__(self, dim, num_experts=8, top_k=2):
        super().__init__()
        self.gate = nn.Linear(dim, num_experts)
        self.top_k = top_k # Default: 2 active experts

    def forward(self, x):
        # Compute gating scores
        scores = self.gate(x)

        # Select top-k experts
        top_k_scores, top_k_indices = torch.topk(
            scores, self.top_k, dim=-1
        )

        # Normalize scores
        top_k_scores = F.softmax(top_k_scores, dim=-1)

        # Create sparse routing tensor
        routing = torch.zeros_like(scores)
        routing.scatter_(-1, top_k_indices, top_k_scores)

        return routing, top_k_indices

```

B V-JEPA Integration

$$\mathcal{L}_{\text{V-JEPA}} = \|\hat{y} - \text{sg}[f_{\theta}(y)]\|^2 \quad (6)$$

$$\mathcal{L}_{\text{MoE}} = \mathcal{L}_{\text{V-JEPA}} + \lambda \cdot \text{LoadBalance}(G) \quad (7)$$

Where LoadBalance ensures even expert utilization across the batch.